

Automotive Cluster Dashboard Report

Multi-Line Packet Parsing and Deployment Verification on BeagleBone Black

Date: June 24, 2026

Target Hardware: BeagleBone Black (ARMv7, Debian 13)

Platform: Qt 6.11.1 (VNC Back-end)

Status: Verified & Fully Functional

1. Executive Summary

This report documents the diagnostic, correction, and deployment processes applied to resolve screen updating issues for the **Automotive Cluster GUI** running on the BeagleBone Black (BBB) target hardware. The system is designed to receive automotive state values (such as speed, battery level, warnings, and gear) via serial Bluetooth transmission, forwarding them to a local Qt-based display application via a Unix domain socket. Multiple updates transmitted as a single batched payload are now successfully parsed, logged, and visualized.

2. Problem Description

Three main issues were preventing correct data visualization on the GUI dashboard:

Issue A: Multi-Line Literal Parsing Failure

The client app transmitted batched command macros (e.g. `battery:15\nthrottle:12\nncruise:0`) with literal backslash-n character sequences (`\n`) rather than actual binary control characters. The socket server was unable to split these, reading them as a single corrupted command string.

Issue B: Active Executable File Lock

Attempts to deploy newly built ARM binaries failed with `Text file busy` errors because the dashboard application was actively running in the background, locking the executable file.

Issue C: USB Power Brownout

High CPU load on the single-core BeagleBone Black processor during multiple

concurrent compilation, file-writing, and service-restart tasks caused a USB current brownout, causing the board to reboot and temporarily disconnect the network.

3. Implementation Details

To resolve the parsing issue, the C++ local socket server receiver was updated to handle both literal text escapes and control character newlines. Below is the code diff of the correction applied to `LocalSocketServer.cpp`:

```
// Read all available text data from the socket

QByteArray data = clientSocket->readAll();

+qDebug() << "Raw data received on socket:" << data;

QString text = QString::fromUtf8(data);

// Replace literal backslash-n and backslash-r with actual newlines/carriagereturns

+text.replace(QStringLiteral("\\n"), QStringLiteral("\n"));

+text.replace(QStringLiteral("\\r"), QStringLiteral("\r"));
```

```
// Split by real newlines to process commands individually
```

```
QStringList lines = text.split(QChar("\n"));
```

```
for (const QString &rawLine : lines) {
```

```
    QString line = rawLine.trimmed();
```

```
    if (line.isEmpty()) continue;
```

```
    // Commands must be in "key:value" format
```

```
    int colonIdx = line.indexOf(':');
```

```
-    if (colonIdx == -1) continue;
```

```
+    if (colonIdx == -1) {
```

```
+        qDebug() << "Command missing colon:" << line;
```

```
+        continue;
```

```
+    }
```

```
QString key = line.left(colonIdx).trimmed().toLower();
```

```
QString valStr = line.mid(colonIdx + 1).trimmed();
```

```
+ qDebug() << "Parsed command key:" << key << "valStr:" << valStr;
```

4. Deployment and Verification

The compiled binary was deployed using an automated Python script that safely stops the systemd services (unlocking the binary file), transfers the executable and configurations, reloads systemd, and restarts the services. Below are the actual execution and parsing logs from the BeagleBone Black system journal:

Test Case	Raw Received String	Parsed Commands & Status
Test 1 (Status Change)	"throttle:0\\nbrake:1\\ngear:P\\nseatbelt:1\\ndoorl:1\\n"	Parsed throttle=0, brake=1, gear=P, seatbelt=1, doorl=1. Updates rendered on screen.
Test 2 (Cruising Mode)	"gear:D\\nthrottle:65\\ncruise:1\\n"	Parsed gear=D, throttle=65, cruise=1. Tachometer and cruise indicators active.
Test 3 (Warning Overlays)	"tire:1\\nbrake:1\\nthrottle:15\\n"	Parsed tire=1, brake=1, throttle=15. Left tire warning light illuminated on cluster

Test Case	Raw Received String	Parsed Commands & Status
UI.		

Verification Conclusion

The systemd logs confirm that the bt-serial-gateway.service establishes a stable connection to /home/debian/dashboard.sock. Grouped command structures are successfully split, parsed, and updated in DashboardAPI, showing instant updates on the GUI display.